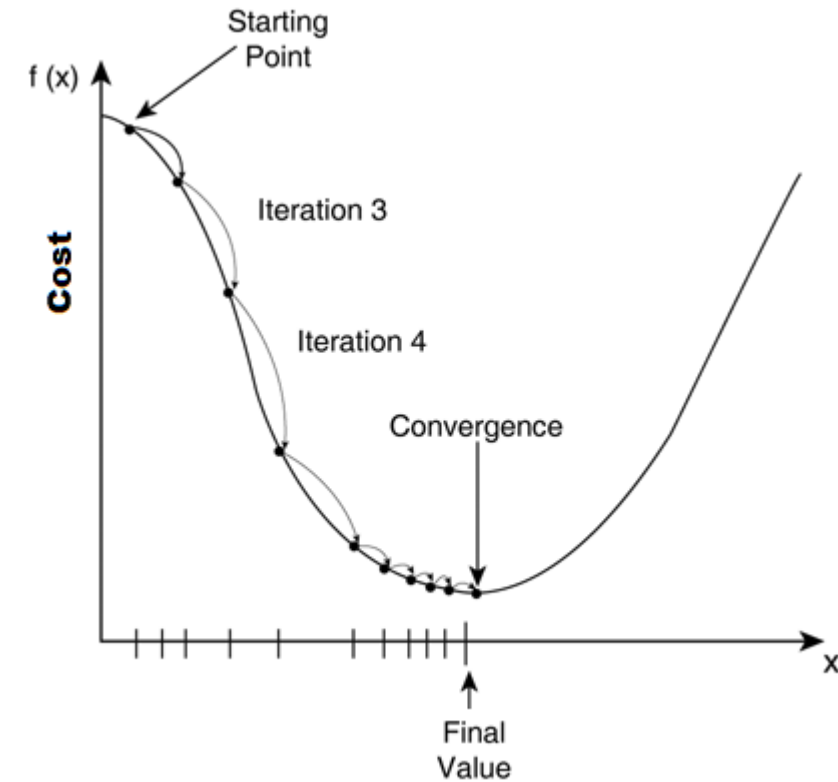


CSE4006

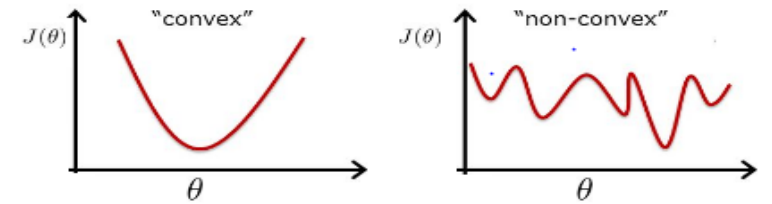
DEEP LEARNING

Gradient based Learning



Gradient based Learning - Introduction

- Designing and training a neural network is not much different from training any other machine learning model with gradient descent.
- The largest difference between the linear models, and neural networks is the **non-linearity** of a neural network.
- Non-linearity causes **loss functions** to become **non-convex**.
- Neural networks are usually trained using iterative, gradient-based optimizers that **drive** the cost function to a **very low value**, rather than the **linear equation** solvers used to train linear regression models or the convex optimization algorithms with guaranteed global **convergence** (e.g., logistic regression or SVMs).
- Convex optimization converges starting from any initial parameters.
- **Stochastic gradient descent** applied to **non-convex** loss functions does **not** have such convergence **guarantee**, and is sensitive to the values of the initial parameters.

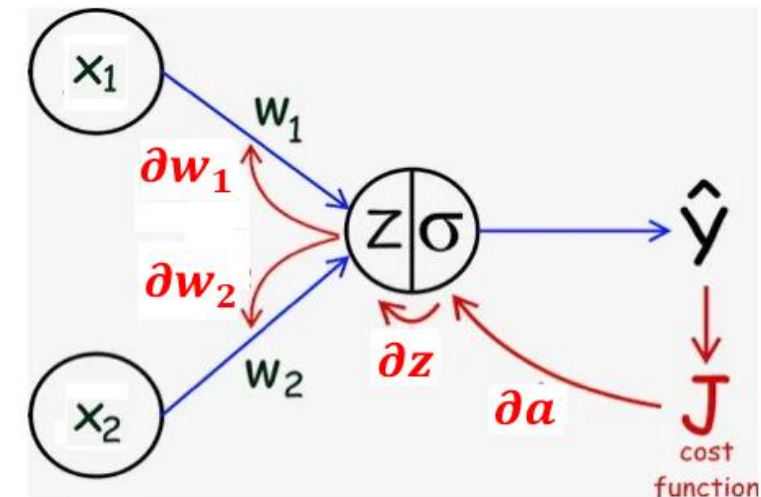


Gradient based Learning - Introduction

- For feedforward neural networks, initialize **all weights** to **small random values**.
- The **biases** may be initialized to **zero** or **small** positive values.
- The training phase is always using the gradient to descend the cost function.
- Linear regression and SVM models can train using gradient descent for **larger** training set.
- Computing the gradient is slightly **more complicated** for a neural network than linear models, but can still be done efficiently and exactly.
- Compute the gradient $(\frac{\partial J}{\partial w_i})$ using the **back-propagation** algorithm.

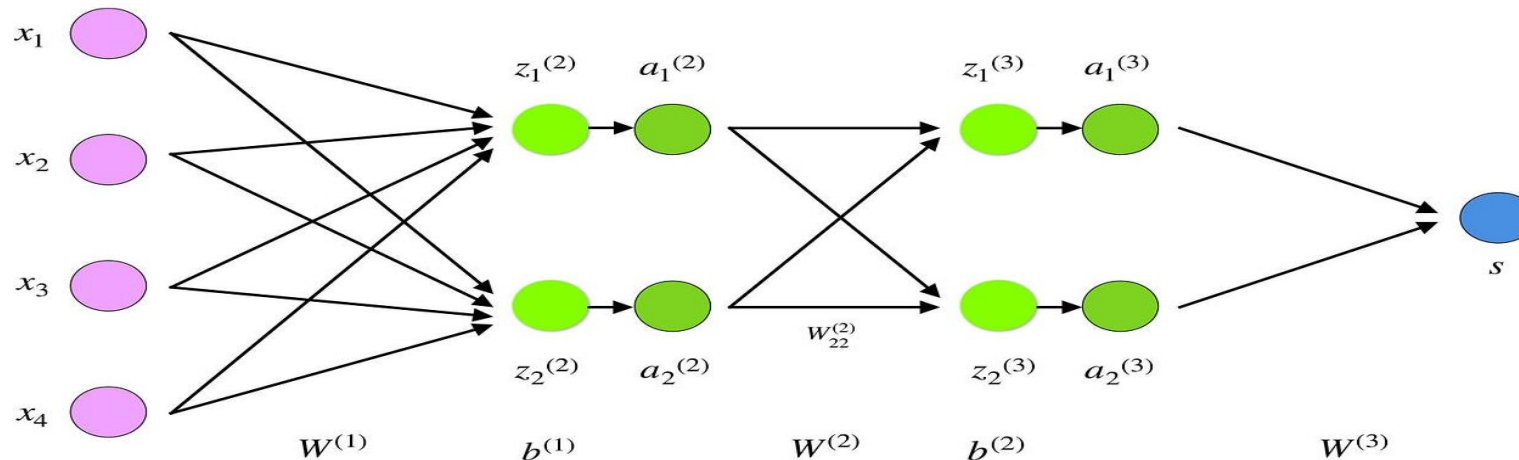
To apply gradient-based learning, **Choosing**

1. The **cost** function, and
2. The form of the **output units**.
3. The **optimizer**



Feed Forward Neural Network - Recap

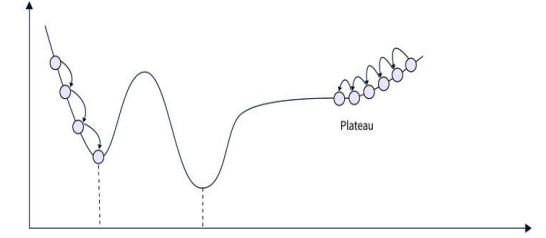
- **Hidden** layers were introduced between the input and the output layers.
- **Activation functions** were used in the hidden layers and the **choice of activation** functions could be based on the problem domain.
- Design the architecture of the network by deciding the number of layers, Connectivity of layers, and Number of (neurons) units in each layer.



- Learning in deep neural networks requires **computing the gradients** of complicated functions.
- **Back-propagation algorithm** can be used to efficiently **compute these gradients**.

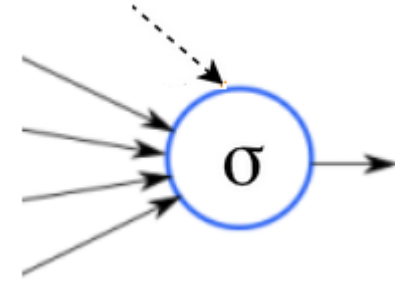
Cost function

- To apply gradient-based learning cost function must be chosen, and additionally what output should be obtained also must be chosen
- The **total cost function** is used to train a neural network.
- Most modern neural networks are trained using **maximum likelihood** i.e., the cost function is simply the **negative log-likelihood** (cross-entropy between the training data and the model distribution).
- The gradient of the **cost function** must be **large**, and able to predict enough for learning algorithm.
- Functions that **saturate** (become very flat) undermine this objective because they make the **gradient** become **very small**.
- Activation functions used to produce the output of the hidden units or the output units **saturate**.
- The **negative log-likelihood** helps to **avoid this problem** for many models.



Maximum Likelihood Estimation

- Objective is to find **theta** that *maximizes the likelihood* of observing the data.
- where $\hat{y}$ is the predicted probability of the positive class, and sigma is some **non-linear activation function** that maps value from $(-\infty, \infty)$ to $[0, 1]$. $\hat{y}_i = \sigma(f(x_i))$
- **Likelihood** = $P(D|\theta) = \prod_{i=1}^n \hat{y}_i^{y_i} * (1 - \hat{y}_i)^{(1-y_i)}$
- **Log-likelihood**
- To check predictions accuracy, verify the predicted probabilities assigned to the correct labels.



$$P(D|\theta) = \sum_{i=1}^m (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

- $\log()$ – **natural log** (base e) logarithm.
- y represents binary class values either zero or one.
- Hence for each index i , model adding either $\log(\hat{y}_i)$ or $\log(1 - \hat{y}_i)$.
- $\hat{y}_i$ - the predicted probability of the i^{th} data point being **positive** class.
- $(1 - \hat{y}_i)$ - the predicted probability of the i^{th} data point being **negative class**.

Procedure to compute log likelihood

- Start with predicted probabilities for the **positive class** ($\hat{y}$).
- If we were given **raw prediction** values, **apply sigmoid** to make it a probability.
- Compute the probabilities for the **negative class** ($1-\hat{y}$).
- Compute the **log** probabilities.
- **Summing** up the log probabilities associated with the true labels.

$$\bullet \hat{y} = \begin{bmatrix} 0.64 \\ 0.27 \\ 0.04 \\ 0.02 \\ 0.81 \end{bmatrix} \quad (1 - \hat{y}) = \begin{bmatrix} 0.36 \\ 0.73 \\ 0.96 \\ 0.98 \\ 0.19 \end{bmatrix} \quad \log(\hat{y}) = \begin{bmatrix} -0.45 \\ -1.31 \\ -3.19 \\ -1.1 \\ -0.21 \end{bmatrix} \quad \log(1 - \hat{y}) = \begin{bmatrix} -1.01 \\ -0.31 \\ -0.04 \\ -0.02 \\ -1.68 \end{bmatrix}$$

$$\bullet y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \hat{y} = \begin{array}{cc} \log(1 - \hat{y}) & \log(\hat{y}) \\ \begin{bmatrix} -1.01 \\ -0.31 \\ -0.04 \\ -0.02 \\ -1.68 \end{bmatrix} & \begin{bmatrix} -0.45 \\ -1.31 \\ -3.19 \\ -1.1 \\ -0.21 \end{bmatrix} \end{array}$$
$$\text{Loss} = y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}) = -3.58$$
$$= -0.45 - 0.31 - 0.04 - 1.1 - 1.68 = -3.58$$

Procedure to compute log likelihood

- The final operation of picking out the correct entries in a matrix is also sometimes referred to as *masking*.
- The mask is constructed based on the **true labels**.

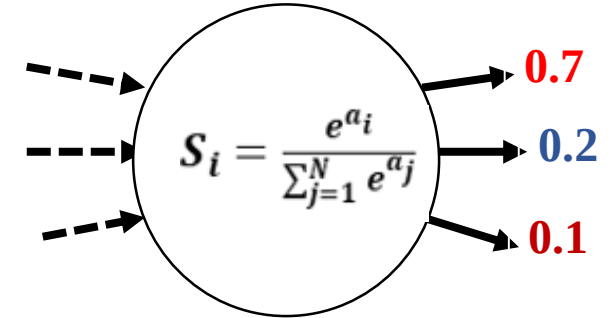
Minimizing the Negative Log-Likelihood

- Apply the **negative** of the **log-likelihood** to minimize the loss is *Negative Log-Likelihood Loss*:
- $J(\theta) = -\sum_{i=1}^m (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$

Cross-Entropy

- Negative log-likelihood is the same as the cross-entropy between y (true labels) and $\hat{y}$ (predicted probabilities of the true labels).

$$h(y, \hat{y}) = - \sum_{i=1}^m (y_i \cdot \log(\hat{y}_i))$$



- **Cross Entropy Loss** applies a **softmax** activation followed by a log transformation.
- But **Negative Log-Likelihood** does not applied.

Generalizing to Multiclass

- Based on “masking principle”, rewrite the **log-likelihood**

$$\log P(D|\theta) = \sum_{i=1}^m \log \hat{y}_i^{(y_i)} = \sum_{i=1}^m (y_i * \log \hat{y}_i)$$

WHEN TO USE Negative Log Likelihood Loss, BCE and CE?

- $Loss(\hat{y}, y)$, where $\hat{y}$ is the prediction values or some transformed version of hypothesis, and y is the label.
- Apply **BCE** Loss if $\hat{y}$ is the probability of a data point being positive.
- Apply **Negative Log-Likelihood Loss**, and **Cross Entropy Loss** when $\hat{y}$ is **two-dimensional** and y is **one-dimensional**, taking values of 0 to $C-1$ with C classes.
- Apply **Negative Log-Likelihood Loss** if $\hat{y}$ encodes **log-likelihood** (it essentially performs the masking step followed by mean reduction).
- Apply Cross Entropy Loss if $\hat{y}$ encodes **raw prediction** values that need to be activated using the **softmax** function.

Choosing Output Units – Gradient Based Learning

- The choice of **cost function** is **tightly coupled** with the choice of **output unit**.
- Mostly, user simply use the cross-entropy between the data distribution and the model distribution.
- **Determine** the form of the cross-entropy function (Binary / Categorical) based on output representation.
- Neural network unit can be used as an output and also used as a hidden unit.

1. Linear Units for Gaussian Output Distributions

- Linear Unit is simple output unit based on an affine transformation without non-linearity.
- Given features h , a layer of linear output units produces a vector $\hat{y} = w^T \cdot x + b$.
- **Linear output layers** are often used to produce the **mean** of a **conditional Gaussian distribution**: $p(y | x) = \mathcal{N}(y; \hat{y}, I)$.

Choosing Output Units – Gradient Based Learning

2. Sigmoid Units for Bernoulli Output Distributions

- Choose it for Binary Classification problems
- The maximum-likelihood approach defines a Bernoulli distribution over y conditioned on x .
- A Bernoulli distribution is defined by just a **single number(Scalar)**.
- The neural net needs to predict only $P(y = 1 | x)$.
- For this number to be a valid probability, it must lie in the interval $[0, 1]$.

$$y' = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = w^T \cdot x + b.$$

Choosing Output Units – Gradient Based Learning

3. Softmax Units for Multinoulli Output Distributions

- To represent a probability distribution over a discrete variable with **n possible classes**.
- More rarely, softmax functions can be used inside the model itself, to choose between one of n different options for some internal variable.
- In the case of binary variables, we wished to produce a single number

$$\textit{Softmax}_i = \frac{e^{a_i}}{\sum_{j=1}^N e^{a_j}} \forall j \in 1..N$$

Choosing Optimizers – Gradient Based Learning

- Gradient Descent
- Stochastic Gradient Descent
- Mini-batch Gradient Descent
- Adagrad
- RMSProp
- Adam
- Adadelta

<https://www.analyticsvidhya.com/blog/2021/03/variants-of-gradient-descent-algorithm/>

Issues in GD

<https://www.scaler.com/topics/momentum-based-gradient-descent/>

Example

<https://www.geeksforgeeks.org/intuition-of-adam-optimizer/>

<https://optimization.cbe.cornell.edu/index.php?title=Adam>

brief

<https://towardsai.net/p/l/impact-of-optimizers-in-image-classifiers>